

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas.

Materia: Programación I

Alumnos:

- Montivero Aaron Luciano
- Melana Colavita Maria Luciana

Grupo N°120

Fecha de entrega: 15/06/2026

INDICE

Marco Teórico -----	Pag 3
Objetivo-----	Pag 3
Diseño del caso Practico-----	Pag 3
Metodología utilizada-----	Pag 5
Resultados-----	Pag 5
Conclusión-----	Pag 7
Repositorio Github-----	Pag 7
Bibliografía-----	Pag 8

Marco teórico

Para el desarrollo de softwares orientados en la gestión de datos, la elección del lenguaje de programación es un factor muy importante, ya que un lenguaje inadecuado puede condenar un proyecto al fracaso desde el principio¹. En este sentido Python se consolida como uno de los lenguajes más utilizado en ciencia de datos debido a su sintaxis limpia y la legibilidad del código.

Dentro de Python podemos manipular la información a través de diferentes estructuras como lo son las listas y diccionarios. Según la documentación oficial de Python² las listas son colecciones de elementos, mutables y ordenadas (conservando de forma estricta el orden posicional a diferencia de los sets). Pueden agrupar secuencias homogéneas o heterogéneas, y facilitan el ordenamiento y recorrido de los datos.

Pero cuando nos enfrentamos a entidades complejas con múltiples atributos, las estructuras secuenciales como las listas son limitadas y propensas a errores. Allí aparecen los diccionarios, que se implementan con esquemas de emparejamiento clave-valor (organizados internamente como tablas), garantizando rapidez en la organización y búsqueda³.

Para garantizar la persistencia de datos, y que los mismos no se pierdan en la volatilidad de la memoria RAM, aparecen los archivos CSV o valores separados por coma. Consiste en básicamente un archivo de texto plano que organiza la información de forma tabular (filas y columnas). Su principal característica es que permite el intercambio de grandes volúmenes de datos de forma ligera sin necesidad de creación de pesadas bases de datos

Objetivos del trabajo

El objetivo del presente proyecto fue desarrollar una herramienta en consola en Python que permita gestionar datos de países utilizando estructuras como listas y diccionarios, almacenadas en un archivo csv, para su filtro y análisis, permitiendo además su guardado y persistencia.

Diseño del caso práctico

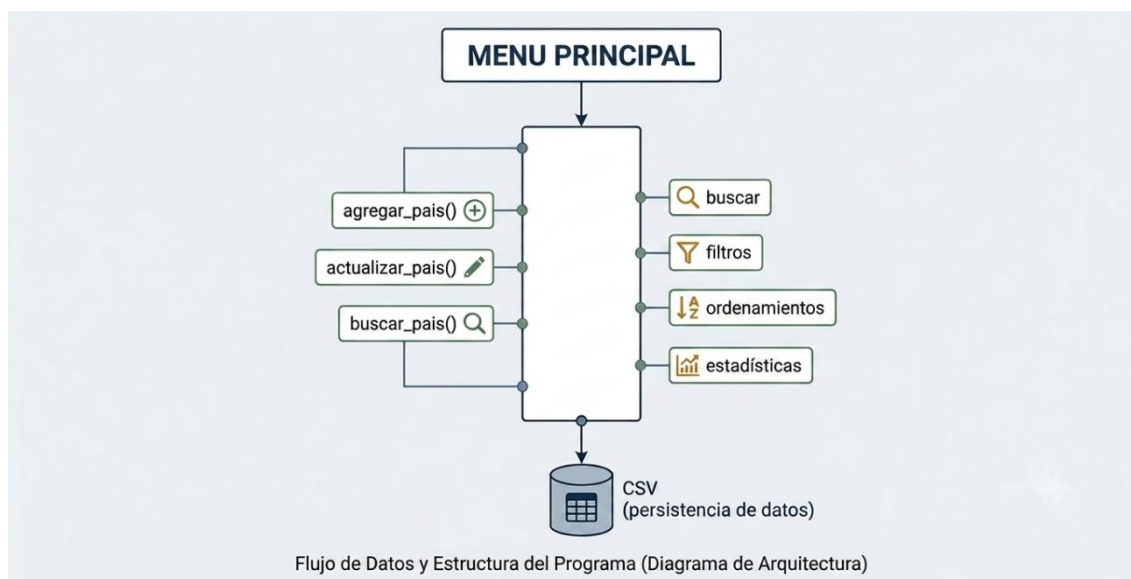
El diseño del código se realizó de forma modular, separando interfaz de usuario, el procesamiento de los datos, y el almacenamiento en CSV

Dentro de la lista global se definió un modelo basado en diccionarios anidados. Cada país cuenta con una estructura compuesta por 4 atributos: nombre (string), población (numérico entero), superficie (numérico float), continente al que pertenece (string).

```
Ejemplo: {  
    'nombre': 'Argentina',  
    'población': 46000000,  
    'superficie': 2780400,  
    'continente': 'América del Sur'  
}
```

El programa se comanda a través de un menú de opciones y un ciclo principal controlado por el usuario. La jerarquía del flujo se divide en tres niveles:

1. **Menú principal o interfaz:** captura la petición del usuario y lo deriva a la función correspondiente
2. **Funciones de control:** formada por bloques independientes que manipulan la lista de diccionarios. Aquí se ejecutan las altas, modificaciones, búsquedas, filtros, ordenamientos y cálculos estadísticos.
3. **Persistencia de datos (CSV):** al iniciar el programa, se lee el archivo externo y así reconstruir el estado de memoria. AL confirmar cambios, se actualiza el archivo de manera persistente



La imagen muestra una interfaz de desarrollo (IDE) con un editor de código y una terminal. El editor muestra el archivo `main.py` con las siguientes líneas de código:

```
1 from helpers import imprimir_error
2 from datos import leer_datos
3 from funcionalidades import (
4     agregar_pais,
5     actualizar_pais,
6     buscar_pais,
7     filtrar_paises,
8     menu_ordenamientos,
9     mostrar_estadisticas,
10 )
11
12
```

La terminal muestra el resultado de la ejecución del programa, que es un menú interactivo:

```
--- MENU ---
1. Agregar pais
2. Actualizar pais
3. Buscar pais
4. Filtrar paises
5. Ordenar paises
6. Estadísticas
7. Salir
Elegí una opción: █
```

Fig 1. Ejecución de código. Menú principal

```

1 from helpers import imprimir_error
2 from datos import leer_datos
3 from funcionalidades import (
4     agregar_pais,
5     actualizar_pais,
6     buscar_pais,
7     filtrar_paises,

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

C:\Users\HP\Downloads\trabajo>C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Downloads/trabajo/main.py

--- MENU ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Salir
Elegí una opción: 3
Ingrese el nombre del país a buscar: argentina

País encontrado:
Nombre: Argentina
Continente: América
Población: 47032768
Superficie: 2780400

```

Fig 2. Búsqueda en base de datos 1

```

1 from helpers import imprimir_error
2 from datos import leer_datos
3 from funcionalidades import (
4     agregar_pais,
5     actualizar_pais,
6     buscar_pais,
7     filtrar_paises,

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

--- ORDENAR ---
1. Nombre
2. Población
3. Superficie
Elegí una opción: 1
País: Alemania | Continente: Europa | Población: 84552242 | Superficie: 357022
País: Angelia | Continente: África | Población: 47435312 | Superficie: 2381741
País: Argentina | Continente: América | Población: 47032768 | Superficie: 2780400
País: Australia | Continente: Oceanía | Población: 27309400 | Superficie: 7692024
País: Brasil | Continente: América | Población: 203080756 | Superficie: 8515767
País: Canadá | Continente: América | Población: 41528880 | Superficie: 9984670
País: China | Continente: Asia | Población: 1409670000 | Superficie: 9596961
País: Corea del Sur | Continente: Asia | Población: 51717480 | Superficie: 100210
País: Egipto | Continente: África | Población: 118365995 | Superficie: 1002450
País: España | Continente: Europa | Población: 49077684 | Superficie: 505990
País: Estados Unidos | Continente: América | Población: 340110988 | Superficie: 9833517
País: Fiyi | Continente: Oceanía | Población: 933154 | Superficie: 18274
País: Francia | Continente: Europa | Población: 68648000 | Superficie: 551695
País: India | Continente: Asia | Población: 1463865525 | Superficie: 3287263
País: Indonesia | Continente: Asia | Población: 285721236 | Superficie: 1904569

```

Fig 3. Ordenamiento por nombre

Metodología Utilizada

Para el desarrollo de este proyecto, se optó por una arquitectura basada en las herramientas integradas de Python, priorizando la eficiencia en la manipulación de los datos y la modularidad del código.

El almacenamiento principal de la información se llevó a cabo mediante listas, las cuales agrupan la totalidad de los países importados. Para la representación individual de cada uno de ellos, se utilizaron diccionarios. A través de la estructura de clave-valor se modelaron los atributos específicos de cada nación, tales como su nombre, población, superficie y continente, facilitando un acceso rápido y ordenado.

La carga y el almacenamiento de los datos se gestionó a través de archivos en formato CSV. Esto asegura un manejo de datos en forma de tabla, sencilla y compatible, permitiendo que la información se conserve entre las distintas ejecuciones del programa.

Con el objetivo de mantener un código limpio y escalable, la lógica del sistema se dividió en funciones específicas y reutilizables. De este modo, tareas como el alta, búsqueda o el filtrado de registros quedan encapsuladas, mejorando la mantenibilidad del software.

El análisis de los datos se apoyó en herramientas nativas del lenguaje. Por un lado, se recurrió a la función `sorted` para el ordenamiento dinámico de los países bajo diferentes criterios (superficie, población o denominación). Por el otro, se integraron funciones como `max`, `min` y `sum`, optimizando el cálculo de métricas estadísticas clave como los extremos demográficos y la media poblacional.

Resultados obtenidos

Se procedió a realizar las pruebas sobre el sistema:

1. Aspectos exitosos del proyecto

El desarrollo modular del proyecto se muestra como uno de los puntos más fuertes. La separación entre `leer_datos` y `escribir_datos` respecto de las funciones operativas permitió realizar pruebas individuales sin riesgo de corromper la estructura principal.

El uso de herramientas como `dictReader` y `dictWriter` permitió mapear por filas al archivo CSV como un diccionario, reduciendo la cantidad de líneas de código necesarias para procesar la información, facilitando que funciones más avanzadas como `mostrar_estadistica()` puedan trabajar de forma más ordenada y directa usando los datos guardados en la memoria.

2. Desafíos

Lo más difícil fue el problema que el formato CSV almacena la información en forma de strings, haciendo que la población y superficie no tuvieran su formato numérico original. Esto generó problemas en el ordenamiento y cálculo de estadística. Para solucionar dicho inconveniente se utilizaron funciones cortas (lambdas) y filtros para convertir strings a números enteros antes de cada cálculo.

3. Gestión de errores

Para garantizar la robustez de la aplicación y evitar interrupciones durante la ejecución, se implementaron `try-except` y bucles de validación continua (`while True`).

- **Validación de tipos de datos:** Se capturaron excepciones del tipo ValueError al pasar strings a enteros, asegurando que el programa no falle si el usuario ingresa caracteres alfabéticos en campos numéricos (como población o superficie).
- **Validación de lógica de negocio:** Se incorporaron controles para impedir el ingreso de valores negativos y estructuras condicionales para verificar que, al solicitar rangos estadísticos, el valor mínimo no supere al máximo.
- **Persistencia de la interfaz:** En lugar de terminar el programa ante un error, el sistema notifica el fallo mediante una función centralizada de feedback (imprimir_error) y solicita el reingreso del dato de forma iterativa, garantizando una experiencia de usuario fluida y libre de frustraciones.

4. Comunicación del grupo

La dinámica del proyecto estuvo condicionada por factores externos como la alta carga horaria de los integrantes con jornadas de trabajo extensas. Este fue el principal desafío para coordinar los bloques de desarrollo.

Para esto se usaron estrategias de comunicación asincrónicas y división de tareas. Se coordinó a través de canales de mensajería instantánea. La integración fue posible mediante la herramienta github donde también se resolvieron los conflictos de versiones.

Conclusión

Como conclusión podemos decir que este proyecto nos permitió aplicar y consolidar los conocimientos adquiridos en Python hasta el momento (listas, diccionarios, funciones, lectura y persistencia de datos en CSV), demostrando la factibilidad de uso y eficiencia de este lenguaje como herramienta de gestión y análisis de datos.

También el uso de Github nos reafirmó la necesidad del uso de herramientas de control de versiones durante el trabajo colaborativo. A pesar de los desafíos de tiempo y responsabilidades de ambos integrantes la plataforma nos permitió integrar exitosamente cada componente del trabajo, logrando con código funcional y eficiente.

Repositorio github

<https://github.com/AaronMonti/tpi-programacion1-utn>

Link a Video

https://www.youtube.com/watch?v=zhRpSP5i_Vo&feature=youtu.be

Bibliografía

1. Verma, S. K. (3 de enero de 2024). *Choosing the Right Programming Language*. DevOps.com. <https://devops.com/choosing-the-right-programming-language/>
2. Python Software Foundation. (2026). *Sorting Techniques: The Timsort algorithm*. Python Standard Library Documentation. <https://docs.python.org/3/howto/sorting.html>
3. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.